

Handbuch

Beispieltitel
für ein Handbuch

Autor: Vorname Nachname

Matrikel-Nr.: xxxxxxxx

Studiengang: Maschinenbau

Erstprüfer: Prof. Dr. Elmar Wings

Abgabedatum: 31. März 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	iii
Tabellenverzeichnis	v
Abkürzungen	vii
1 Hinweise	1
2 Main Function	3
2.1 Test	3
3 Schrittmotor	5
3.1 Beschreibung eines Schrittmotors	5
3.1.1 Aufbau und Funktionsweise eines Schrittmotors	5
3.1.2 Schrittmotor Bauformen	6
3.1.3 Betriebsarten unipolar und bipolar	7
3.1.4 Mikroschrittverfahren	7
3.2 Schrittverluste verhindern	8
3.2.1 Auswahl des Schrittmotors	8
3.2.2 Betriebsart	8
3.2.3 Externe Ereignisse	10
3.3 Beschreibung des verwendeten Schrittmotors	10
3.4 Beschreibung des Programms auf dem Arduino	11
3.4.1 Aufgabe des Programms	11
3.4.2 Erklärung des Codes	11
3.5 Bibliotheken	17
3.5.1 Accelstepper	17
3.5.2 Wire	18
3.5.3 Adafruit GFX	18
3.5.4 Adafruit SSD1306	18
3.5.5 Encoder	19
3.6 Schrittmotorsteuerung	19
3.7 Schrittverluste verhindern	20
3.8 Auswahl des Schrittmotors	20
3.9 Betriebsart	20
3.9.1 Start-Stopp-Betrieb	21
3.9.2 Beschleunigung und Rampenprofil (Trapezförmig)	21

Inhaltsverzeichnis

3.10 Externe Ereignisse	22
3.10.1 Lastrückkopplung	22
3.10.2 Erhöhung der Nutzlast mit der Zeit	22
4 Skizze	27
5 Liste der Teile	29
6 Aufbau	31
7 Funktionen	33
8 Wartung	35
9 Sicherheitshinweise	37
10 Troubleshooting	39
11 Sonstiges	41

Abbildungsverzeichnis

3.6	Treiberkarte Debo DRV A4988	19
3.1	Prinzipieller Aufbau Schrittmotor (2-phasig) [Hagl:2021]	23
3.2	Start-Stopp Frequenz [Faulhaber:2020]	23
3.3	Trapezförmiges Geschwindigkeitsprofil [Faulhaber:2020]	23
3.4	Drehmoment-Diagramm für den Schrittmotor 42BYG015 [Gems:2014]	24
3.5	Flussdiagramm des Programms	25
3.7	Start-Stopp Frequenz [Faulhaber:2020]	25
3.8	Trapezförmiges Geschwindigkeitsprofil [Faulhaber:2020]	25

Tabellenverzeichnis

3.2	Ursachen und Lösungen: Motor beendet die Beschleunigungsrampe nicht [Faulhaber:2020]	9
3.3	Ursachen und Lösungen: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist. [Faulhaber:2020]	9
3.4	Ursachen und Lösungen: Motor läuft nicht an [Faulhaber:2020]	21
3.5	Ursachen und Lösungen: Motor beendet die Beschleunigungsrampe nicht [Faulhaber:2020]	21
3.6	Ursachen und Lösungen: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist. [Faulhaber:2020]	22
3.1	Ursachen und Lösungen: Motor läuft nicht an [Faulhaber:2020]	23

Abkürzungen

1 Hinweise

Bitte verwenden Sie `biber.exe`. Hinweis: Wir waren heute beim Labor

- Welcome - Beschreibung der allgemeinen Funktion
- Sicherheitshinweise
- Überblick
 - Hardwarebeschreibung aus der Sicht des Bedieners
 - Vorbereitung, z.B. Aufstellung des Geräts, Installation, Stromversorgung
 - ...
- Auflistung der Funktionen
 - Funktion 1: Beschreibung jeder einzelnen Funktion aus Sicht des Benutzers
 - Funktion 2: Beschreibung jeder einzelnen Funktion aus Sicht des Benutzers
 - ...
- Mögliche Fehler, Fehlerursachen und deren Behebung
- Technische Spezifikation

2 Main Function

2.1 Test

3 Schrittmotor

In diesem Kapitel folgt eine grundsätzliche Beschreibung eines Schrittmotors. Darauf folgt die Erläuterungen zum Aufbau und Funktionsweise eines Schrittmotors sowie die Aufzählung verschiedener Grundbauarten. Nachfolgend werden Ursachen und Lösungen zum Verhindern von Schrittfehlern und der verwendete Schrittmotor genauer erläutert.

3.1 Beschreibung eines Schrittmotors

Ein Schrittmotor ist ein Elektromotor, der sich für präzise Positionierungsaufgaben eignet. Im Gegenteil zu anderen Elektromotoren wird bei einem Schrittmotor keine Positionsmessung oder Positionsregelung benötigt. Andere mechanische Antriebssysteme benötigen einen geschlossenen Regelkreis und mechanische Bremsen um Drehzahl und Position einzuhalten. Der Motor führt durch die Rotation des Rotors diskrete Schritte aus, wobei jeder Steuerimpuls eine Verschiebung um einen konstanten Winkel bewirkt. Mit diesem Motor ist eine erreichbare Positioniergenauigkeit von $0,1^\circ$ möglich. Diese Art von Elektromotor wird beispielsweise in Druckern oder Scanner, aber auch im Kraftfahrzeugbereich verwendet. Im Kraftfahrzeugbereich werden die Schrittmotoren zur Spiegelverstellung sowie der Sitzverstellung verwendet. Das maximal erreichbare Drehmoment eines Schrittmotors liegt bei zwei Newtonmeter und die maximal erreichbare Drehzahl bei ca. 2000 Umdrehung pro Minute. Der Vorteil eines Schrittmotors ist die Wartungsfreiheit, da der Rotor keine Wicklungen hat. [Babiel:2023; Hagl:2021; Bernstein:2018; Schroeder:2021]

3.1.1 Aufbau und Funktionsweise eines Schrittmotors

Der Schrittmotor besteht aus einem Stator, einem Rotor ohne Wicklungen und einer Steuerelektronik. Diese Steuerelektronik setzt sich zusammen aus einer Treiberstufe und der eigentlichen Steuerung. Bei dem Stator handelt es sich um den feststehenden äußeren Teil und bei dem Rotor um den beweglichen inneren Teil, wie in Abbildung 3.1 zu erkennen ist. Im Stator sind Spulen verbaut, die von einem Strom durchflossen werden. Hierdurch entsteht ein magnetisches Feld. Da der Rotor magnetisch ist, folgt er dem Magnetfeld des Stators. Soll eine Bewegung hervorgerufen werden, werden einzelne Wicklungsstränge ein- aus- oder umgeschaltet. Durch diesen Vorgang wird ein rotierendes Magnetfeld erzeugt. Das erzeugte Magnetfeld zieht den Rotor an. Der Rotor wird bei jedem Puls (Takt) um einen Winkelschritt weitergeschaltet. Die Anzahl der Polpaare im Stator geben die Anzahl der Schritte vor. Die Drehzahl und

3 Schrittmotor

die Drehrichtung hängt von der Reihenfolge und der Häufigkeit der Stromimpulse ab. Es gibt drei verschiedene Betriebsarten, die abhängig von der Genauigkeit und der Drehzahl sind. Im Vollschrittbetrieb werden alle Polpaare bestromt. In dem Halbschrittbetrieb wird die Schrittzahl des Motors verdoppelt, wodurch sich die Positionsaufösung im Gegensatz zum Vollschrittbetrieb verdoppelt. Allerdings wird in diesem Betrieb das Drehmoment reduziert. In dem Mikroschrittbetrieb bewegt sich der Rotor in sehr kleinen Schritten. Hierdurch wird eine hohe Positioniergenauigkeit und ein ruhiger Lauf erreicht, da die Ströme und das Drehmoment in kleineren Schritten verändert werden. Bei einer zu hohen Schrittfrequenz kann ein Schrittverlust entstehen. Bei einem Schrittverlust überspringt der Motor einzelne Winkelschritte und landet in einer vorherigen oder nächsten Position gleicher Phase. Durch die offene Steuerkette werden die Verluste nicht erkannt.[Hagl:2021; Bernstein:2018; Schroeder:2021]

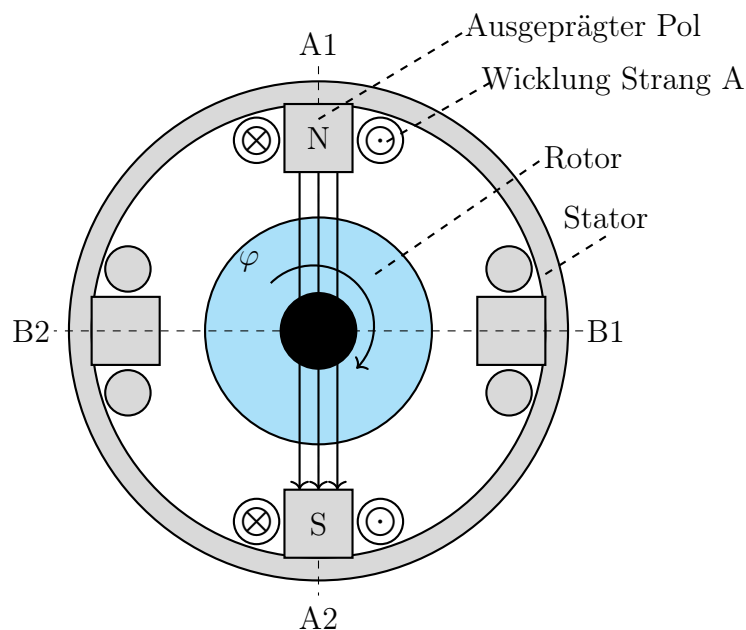


Abbildung 3.1: Prinzipieller Aufbau Schrittmotor (2-phasig) [Hagl:2021]

3.1.2 Schrittmotor Bauformen

Wie bei anderen Elektromotoren gibt es auch bei dem Schrittmotor verschiedene Grundbauarten.

- Permanentmagnetenerreger-Schrittmotor (PM-Schrittmotor)
- Reluktanzschrittmotor (VR-Schrittmotor)
- Hybridschrittmotor (Hybrid-Schrittmotor)

Der **permanentmagnetische Schrittmotor** hat einen Permanentmagneten in dem Rotor verbaut. Hierbei stellt sich der permanentmagnetische Rotor immer so, dass der Nordpol des Rotors dem Nordpol des Statorfeldes gegenüber liegt und der Südpol des Rotors dem Südpol des Statorfeldes. In dieser Ausrichtung ziehen sich die Pole gegenseitig an. Die Drehrichtung des Rotors hängt von der Fließrichtung des Stromes ab. Der permanentmagnetische Schrittmotor entwickelt im ausgeschalteten Zustand ein Drehmoment zur Selbsthaltung. Dies ist aufgrund des permanentmagnetischen Rotors möglich. Bei diesem Drehmoment handelt es sich um das höchste Drehmoment, das auf die Welle des Motors übertragen werden kann, ohne dass diese sich in eine rotierende Bewegung versetzt. Zu dieser Art von Schrittmotoren gehören beispielsweise der Klauenpol-Schrittmotor und der Scheibenmagnet-Schrittmotor. Bei der zweiten Bauart handelt es sich um den **Reluktanzschrittmotor**. Bei dieser Bauart besteht der Rotor aus einem weichmagnetischen Material und besitzt eine gezahnte Form. So lange der Schrittmotor von keinem Strom durchflossen wird, entsteht kein Magnetfeld. Sobald der Motor in Betrieb genommen wird, entsteht ein magnetischer Fluss innerhalb des Rotors. Wird nun eine Wicklung erregt, wird der nächste Zahn des Rotors angezogen. Dadurch, dass die Rotorzähne ungleich der Polteilung sind, kann das System unendlich lange fortgesetzt werden. Die Anzahl der Schritte und die Genauigkeit des Reluktanzschrittmotors ist abhängig von der Anzahl der Zähne auf dem Rotor. Aus technischer Sicht sind mit dieser Bauart Schrittwinkel unter 1° möglich. Damit die Drehrichtung verändert werden kann, sind mindestens zwei Strangwicklungen nötig. Bei den **Hybridschrittmotoren**, auch bekannt als Hybridschrittmotoren handelt es sich um eine Kombination aus dem Reluktanzschrittmotor und dem Permanentmagneterreger-Schrittmotor. Durch diese Kombination aus den beiden Schrittmotoren werden die Vorteile aus der kleinen Schrittweite, dem hohen Drehmoment und dem Selbsthaltungemoment genutzt. Bei dem Hybridschrittmotor besteht der Rotor aus zwei um eine halbe Zahnteilung versetzten weichmagnetischen Polrädern, die eine zahnförmige Form haben. Bei den beiden Polrädern bildet das eine Polrad den Nordpol und das zweite Polrad den Südpol. Zwischen den beiden Polrädern befindet sich ein Permanentmagnet. Anders als bei anderen Schrittmotoren wird der Rotor bei dieser Bauart axial magnetisiert. Damit ein kleiner Schrittwinkel möglich ist, haben die Statorpole ebenfalls eine zahnförmige Form. Die Ausrichtung des Rotors ist abhängig von der Stromrichtung und wird durch den minimalen Widerstand bestimmt, der sich aus dem Stromfluss durch die einzelnen Stränge ergibt. Wird ein besonders kleiner Schrittwinkel benötigt, kann dies durch Erhöhung der Zähnezahl erreicht werden. [Schroeder:2021; Hagl:2021; Babel:2023]

3.1.3 Betriebsarten unipolar und bipolar

Neben den oben bereits genannten Betriebsarten, kann außerdem zwischen dem Unipolarbetrieb und dem Bipolarbetrieb unterschieden werden. Der große Unterschied zwischen den beiden Betriebsarten besteht darin, dass in dem Unipolarbetrieb der Strom in eine Richtung fließt. Bei dem Bipolarbetrieb hingegen fließt der Strom in beide Richtungen. Dies ist möglich, da jeder Wicklungsstrang über eine Vollbrücke

3 Schrittmotor

gespeist wird. Ein weiterer Unterschied besteht in der Schaltung der Zweige, durch die ein Gleichstrom fließt. In dem Unipolarbetrieb werden die beiden Zweige in Reihe geschaltet. Jeder Wicklungsstrang wird mit zwei Drähten parallel gewickelt. Sind die beiden Wicklungsstränge in dem Bipolarbetrieb parallel gewickelt, müssen die Zweige parallelgeschaltet werden. Im Bipolarbetrieb kann ein höherer Wirkungsgrad erzielt werden, wo hingegen der Unipolarbetrieb eine deutlich einfachere Schaltung aufweist. [Schroeder:2021]

3.1.4 Mikroschrittverfahren

Mit dem Mikroschrittverfahren können Zwischenschritte und Mikroschritte erzeugt werden. Es bietet die Fähigkeit, Geräusche und Vibrationen zu minimieren sowie die Auflösung der Umdrehung zu erhöhen. Um Zwischenschritte zu erreichen, müssen beide Wicklungen eines Schrittmotors gleichzeitig mit Strom versorgt werden. Mit dem Sinus/Cosinus-Mikroschrittverfahren können die Mikroschrittverfahren realisiert werden, indem der Strom in jeder Phase des Motors sinusförmig variiert. Durch die Anwendung sinusförmiger Ströme auf die Motorwicklungen wird eine feinere und gleichmäßigere Bewegung erzielt, was zur einer präziseren Steuerung und reduzierten mechanischen Vibrationen führt. [McComb:2024]

Ein Mikroschritt-Treiber unterteilt die Motorschrittwinkel in vier oder mehr Teil. Es handelt sich um einen Controller, der die Methode der Stromreglung verwendet, um die Mikroschritte zu erzeugen. Der Controller bietet außerdem den Vorteil einer erhöhten Flexibilität bei der Integration der Strombegrenzung und der Anpassung der Antriebscharakteristik als Reaktion auf Änderungen der Systemdynamik. Die Anzahl der Unterteilungen ist einstellbar, was eine präzise Steuerung und Anpassung an spezifische Anforderungen ermöglicht. [Babiel:2023]

3.2 Schrittverluste verhindern

Um mögliche Schrittverlusten einzugrenzen und oder sie zu verhindern, wird in diesem Kapitel beschrieben, wie die Ursachen für Schrittverluste oder Stillstand methodisch zu ermitteln sind. [Faulhaber:2020]

3.2.1 Auswahl des Schrittmotors

Zunächst muss ein passender Motor für die Anwendung gewählt werden. Dabei sollten folgende grundlegende Regeln erfüllt sein:

- Motorauswahl durch Höchstwerte für Drehmoment und Drehzahl (Worst-Case-Szenario)
- Verwendung eines Sicherheitsaufschlag von 30 % auf die Drehmoment-Drehzahl-Kennlinie(Kippmoment)
- Sicherstellen, dass externe Ereignisse die Anwendung nicht blockieren können

Sind die geforderten Drehmomente bei den jeweiligen Drehzahlen, den Motorspezifikation entsprechend, dann sind keine Probleme zu erwarten. Ist der Motor zu schwach gewählt und die Anwendung fordert mehr Leistung als der Motor abgeben kann, so bleibt der Motor stehen. Der nächste Schritt ist die Durchführung von Testdurchläufen. Es soll im Betrieb überprüft werden, ob Schritverluste auftreten. Schrittmotoren verlieren konstruktionsbedingt nicht nur einen einzigen Schritt. Bei geringen Drehzahlen verliert der Motor ein Vielfaches von vier Schritten.[Faulhaber:2020]

3.2.2 Betriebsart

In diesem Abschnitt werden je nach Betriebsart mögliche Ursachen erläutert, falls der Schrittmotor bei den Tests versagt.

Start-Stopp-Betrieb

Der Motor ist mit der Last fest verbunden und wird mit konstanter Drehzahl betrieben. Innerhalb des ersten Schrittes muss der Motor auf die vorgegebene Frequenz beschleunigen wie in Abbildung 3.7 zu sehen ist.[Faulhaber:2020]

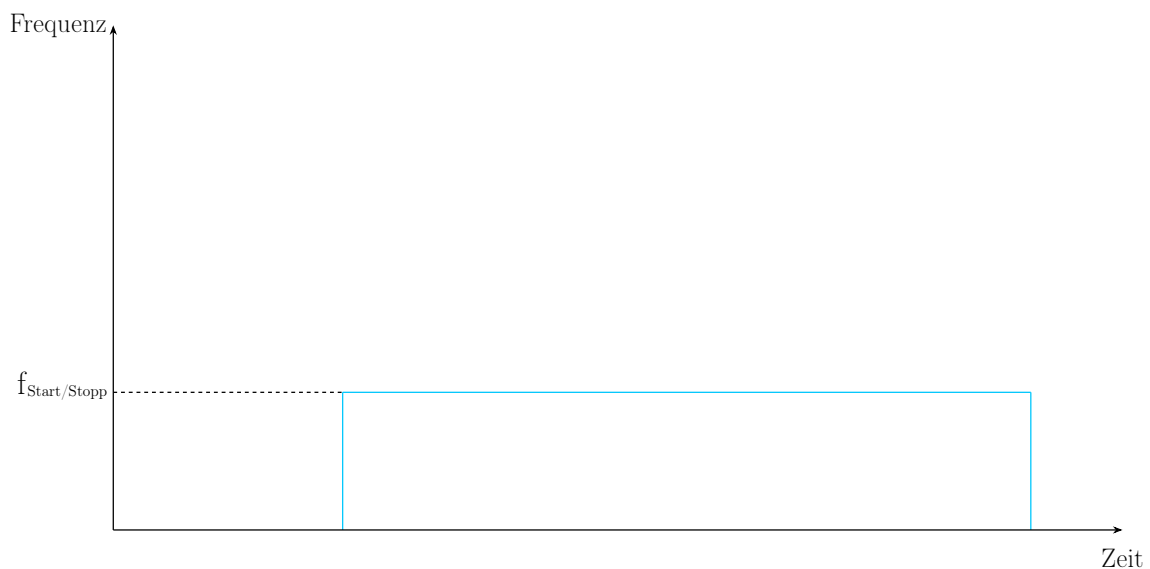


Abbildung 3.2: Start-Stopp Frequenz [Faulhaber:2020]

Fehlerbild: Motor läuft nicht an (siehe Ursachen und Lösungen aus Tabelle 3.1)

Beschleunigung und Rampenprofil (Trapezförmig)

Der Motor kann mit einer im Controller vorgegebenen Beschleunigungsrate bis auf die Maximalfrequenz beschleunigen wie in Abbildung 3.8 zu sehen ist.[Faulhaber:2020]

Fehlerbild: Motor beendet die Beschleunigungsrampe nicht (siehe Ursachen und Lösungen aus Tabelle 3.2)

3 Schrittmotor

Ursachen	Lösungen
Last zu Hoch	Falscher Motor, größeren Motor wählen
Frequenz zu hoch	Frequenz reduzieren
Pendelt der Motor von Links nach Rechts	Es könnte eine Phase unterbrochen oder nicht angeschlossen sein, dies muss repariert werden
Phasenstrom passt nicht	Phasenstrom erhöhen

Tabelle 3.1: Ursachen und Lösungen: Motor läuft nicht an [Faulhaber:2020]

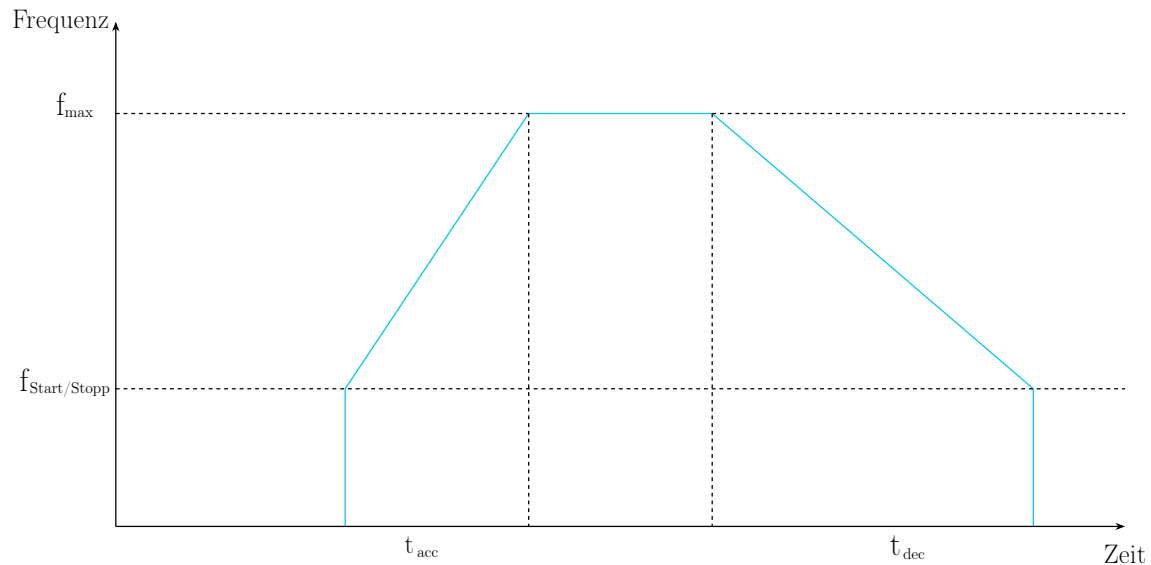


Abbildung 3.3: Trapezförmiges Geschwindigkeitsprofil [Faulhaber:2020]

Ursachen	Lösungen
Motor bleibt bei der Resonanzfrequenz hängen	• Beschleunigung erhöhen, um die Resonanzfrequenz schneller zu durchlaufen • Start-Stopp Frequenz über dem Resonanzpunkt wählen • Halbschritt- oder Mikroschrittbetrieb verwenden • Mechanische Dämpfung vorsehen
Falsche Einstellung von Versorgungsspannung oder Strom zu gering	• Spannung oder Strom erhöhen • Motor mit geringerer Impedanz testen • Stromregelung verwenden
Maximaldrehzahl zu hoch	• Maximaldrehzahl reduzieren • Beschleunigungsrampe abflachen
Schlechte Vorgabe der Beschleunigungsrampe durch Elektronik	• Anderen Controller verwenden

Tabelle 3.2: Ursachen und Lösungen: Motor beendet die Beschleunigungsrampe nicht [Faulhaber:2020]

Fehlerbild: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist (siehe Ursachen und Lösungen aus Tabelle 3.3)

Ursachen	Lösungen
Motor wird an Leistungsgrenze betrieben und bleibt stehen aufgrund zu hoher Beschleunigung	• Ruckeln verringern durch geringere Beschleunigungsrate oder durch unterschiedliche Beschleunigungsrampen, erst steil dann flacher • Drehmoment erhöhen • Motor im Mikroschrittbetrieb betreiben • Mechanische Dämpfung vorsehen

Tabelle 3.3: Ursachen und Lösungen: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist. [Faulhaber:2020]

3.2.3 Externe Ereignisse

Lastrückkopplung

„Manchmal wird der vom Motor angetriebene Mechanismus/Last während der Bewegung „aufgezogen“ und gibt diese Energie wieder an den Motor zurück, wenn die Ströme ausgeschaltet werden. Der Mechanismus könnte z.B. ein Untersetzungsgetriebe sein.“ [Faulhaber:2020] Wird diese Energie an den Motor zurück geleitet, kann es passieren, dass der Motor sich um einen Winkel verdreht, der mehr als einem Schritt entspricht. Dabei kann es passieren, dass der Motor nicht ausreichend Drehmoment entwickelt und nicht oder erst nach 4 Vollschritten anläuft. [Faulhaber:2020]

Lösungen:

- Die Kommutierung so programmieren, dass Wert und Polarität vor dem Abschalten gespeichert und beim Wiedereinschalten verwendet werden kann.
- Nicht vollständig den Strom abschalten, sondern bei Motorstillstand einen reduzierten *Stand-By* Strom aufrechterhalten

Erhöhung der Nutzlast mit der Zeit

„Manchmal läuft der Motor für eine lange Zeit störungsfrei und viel später treten die ersten Schrittverluste auf. In diesem Fall ist es sehr wahrscheinlich, dass die Last, die der Motor „sieht“, sich geändert hat. Das kann auf Verschleiß der Motorlager oder ein externes Ereignis zurückzuführen sein.“ [Faulhaber:2020]

Lösungen:

3 Schrittmotor

- Prüfen ob ein externes Ereignis durch Veränderung des Mechanismus vorliegt.
- Prüfen ob Lagerverschleiß vorhanden ist. Verwendung von Kugellager erhöhen die Lebensdauer des Motors.
- Verwendung von Schmiermittel um Reibung zu verhindern.

3.3 Beschreibung des verwendeten Schrittmotors

Für das Automatisierungsprojekt wird ein Nema 17 Schrittmotor der Firma Creality3D verwendet. Dieser Schrittmotor wird im 3D-Druck sowie in CNC-Maschinen eingesetzt. Der Motor arbeitet im Bipolarbetrieb und es sind zwei Spulen verbaut. Er hat einen Schrittwinkel von $1,8^\circ$ und benötigt 200 Schritte für eine volle Umdrehung. Die Wellenlänge beträgt 20 mm und der Wellendurchmesser 5 mm. Der ausgewählte Schrittmotor arbeitet mit einer Nennspannung von 12 Volt. Der Motor ist nicht für große Drehmomente ausgelegt, sondern für schnelle und präzise Bewegungen. Dies wird qualitativ in der Abbildung 3.4 für den Schrittmotor GM42BYG015 dargestellt. Hier ist zu erkennen, dass der Schrittmotor ein Drehmoment von ca. $0,8 \text{ kg}\cdot\text{cm}$ erreicht. Auf der x-Achse wird die Geschwindigkeit in Pulses per Second (PPS) angegeben, d.h. die Anzahl der Steuerimpulse pro Sekunde. Der Schrittmotor erzielt aber eine gute Positioniergenauigkeit durch den kleinen Schrittwinkel und die Microstepp-Funktion. Des weiteren arbeitet der Motor mit 0,84 Amper pro Phase. Weitere Vorteile des Schrittmotors sind die kompakte Bauform mit den Abmessung $42 \times 42 \times 34 \text{ mm}$ sowie die geringe Masse mit 220 Gramm. Betrieben werden kann der Schrittmotor in einer Umgebungstemperatur von -10°C bis $+50^\circ\text{C}$. [Gems:2014; Gems:2022]

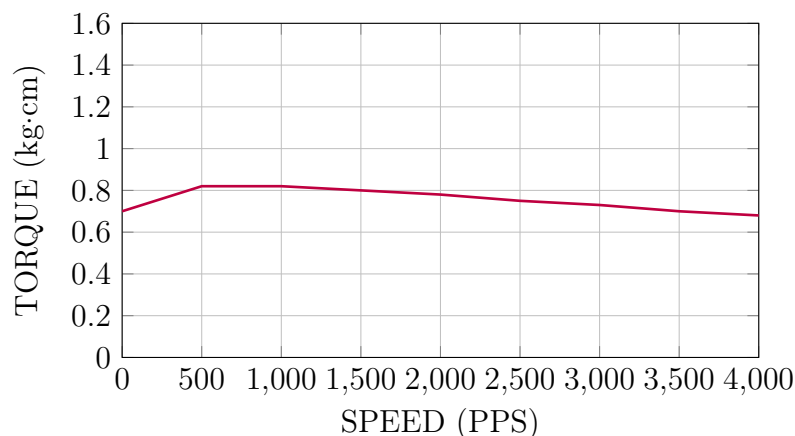


Abbildung 3.4: Drehmoment-Diagramm für den Schrittmotor 42BYG015 [Gems:2014]

3.4 Beschreibung des Programms auf dem Arduino

3.4.1 Aufgabe des Programms

Das Programm steuert einen Schrittmotor und zeigt Informationen auf einem OLED-Display an. Es nutzt einen Drehwinkel-Encoder, um verschiedene Betriebsstufen des Motors auszuwählen.

3 Schrittmotor

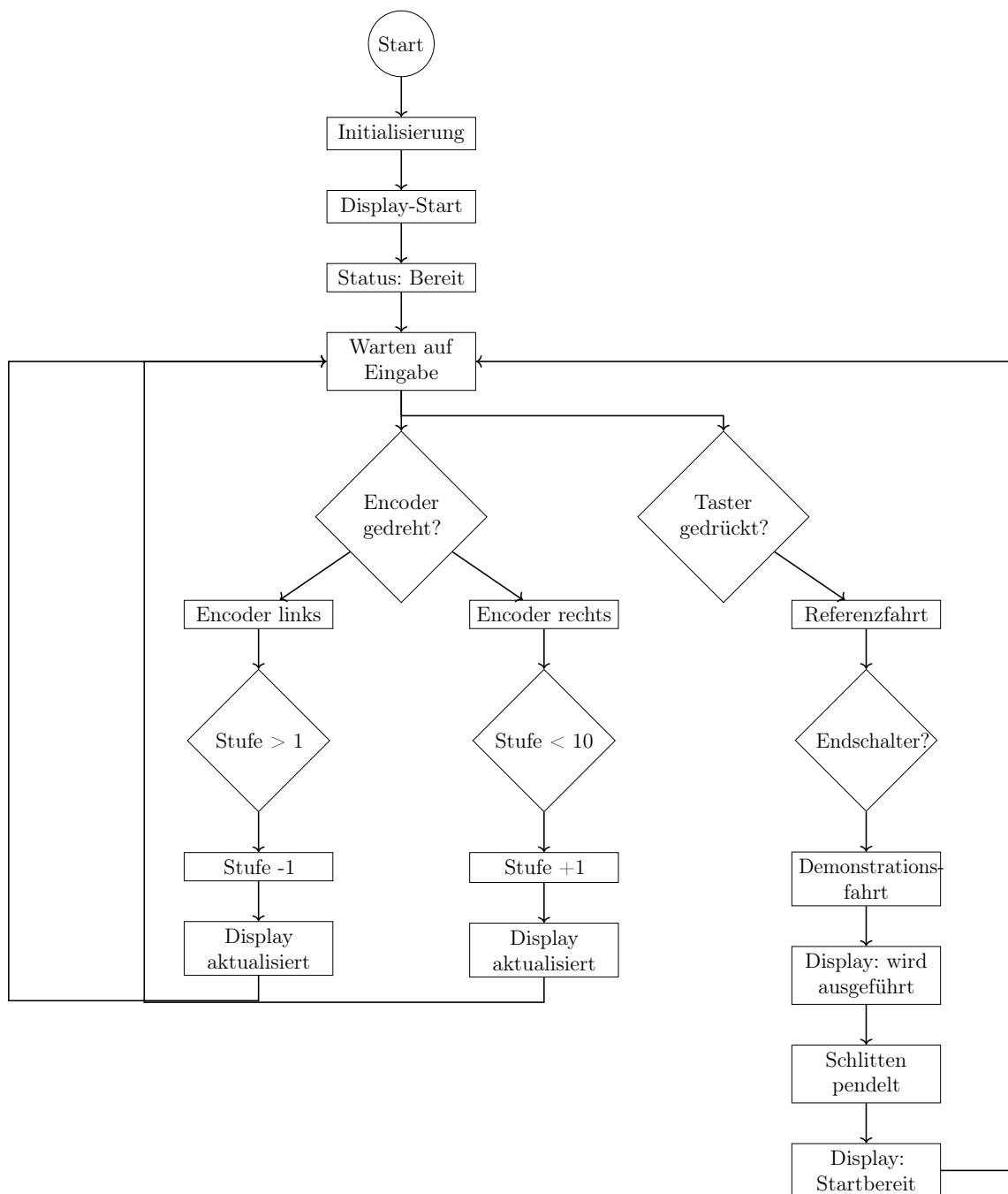


Abbildung 3.5: Flussdiagramm des Programms

Beim Start des Programms werden alle Komponenten, einschließlich des Schrittmotors, des Displays, der LEDs und des Encoders, initialisiert. Nach einer kurzen Startanimation auf dem Display wechselt das System in den Bereitschaftsmodus. Der Benutzer kann den Drehwinkel-Encoder drehen, um eine der zehn verfügbaren Stufen auszuwählen. Jede Stufe hat spezifische Geschwindigkeitswerte für den Schrittmotor.

Die aktuell ausgewählte Stufe wird auf dem OLED-Display angezeigt, sodass der Benutzer immer über die gewählte Einstellung informiert ist. Ein Taster ermöglicht es dem Benutzer, den Startprozess des Schrittmotors zu initiieren. Nach dem Drücken des Tasters führt der Schrittmotor Bewegungen aus, die der ausgewählten Stufe entsprechen. Diese Bewegungen beinhalten das Fahren zu einem festgelegten Punkt und das anschließende Zurückfahren zur Ausgangsposition. Zusammengefasst ermöglicht das Programm dem Benutzer, durch Drehen des Encoders verschiedene Betriebsstufen auszuwählen und den Schrittmotor zu steuern. Die Stufenanzeige auf dem Display und die LED-Statusanzeigen bieten dabei eine klare und leicht verständliche Rückmeldung über den aktuellen Zustand des Systems. Der Ablauf des Programms ist in Abbildung 3.5 dargestellt.

3.4.2 Erklärung des Codes

Das Programm zur Steuerung des Demonstrators wurde zunächst aus Teilen der Testprogramme für jede Hardware-Komponente basierend auf der gewünschten Funktion zusammengefügt. Nachdem die gewünschte Funktionalität erreicht war, wurde der Code mit Hilfe des Large-Language-Models Chat GPT in der Version 4.0 des US-Unternehmens OpenAI Inc. bezüglich der Lesbarkeit und Vereinheitlichung Benennung von Variablen. Zuerst werden die notwendigen Bibliotheken eingebunden, die für die Steuerung des Schrittmotors, das Display und den Encoder erforderlich sind. 3.4.2

```
#include "AccelStepper.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Encoder.h>
```

[Einbindung der Bibliotheken]Einbindung der Bibliotheken

Die define-Anweisungen legen Konstanten fest, wie die Art des Motorinterfaces, die Abmessungen des Displays und die Pin-Nummern für den Stepper-Motor, die LEDs, den Taster, den Endschalter und den Encoder. 3.4.2

```
#define MOTOR_INTERFACE_TYPE 1
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C

#define PIN_STEP_DIR 4
#define PIN_STEP_STEP 5
#define PIN_LED_RED 6
#define PIN_LED_GREEN 3
#define PIN_LED_BLUE 9
#define PIN_BUTTON 11
#define PIN_ENDSTOP 12
```


3 Schrittmotor

```
#define PIN_CLK 9
#define PIN_DT 2
#define PIN_SW 3
```

[define-Anweisungen]define-Anweisungen

Nun werden die Objekte für den Stepper-Motor, das Display und den Encoder erstellt.

```
AccelStepper stepper(MOTOR_INTERFACE_TYPE, PIN_STEP_STEP, PIN_STEP_STEP,
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET,
Encoder encoder(PIN_DT, PIN_CLK);
```

Die globalen Variablen speichern den Status des Tasters, die Zeit der letzten Zustandsänderung, die aktuelle Stufe sowie die Geschwindigkeiten und Beschleunigungen für jede Stufe. 3.4.2

```
int buttonStatus = HIGH;
int lastButtonStatus = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long debounceDelay = 50;
int endstopStatus = HIGH;
int lastEndstopStatus = HIGH;
unsigned long lastEndstopDebounceTime = 0;
int stage = 0;
int speeds[] = {1000,
               500,
               750,
               1000,
               1250,
               1500,
               1750,
               2000,
               2250,
               2500,
               1750};
int accelerations[] = {1000,
                       6000,
                       6000,
                       6000,
                       6000,
                       6000,
                       6000,
                       6000,
                       6000,
                       6000,
                       6000};
long oldPosition = -999;
```

3.4 Beschreibung des Programms auf dem Arduino

```
const int safeSteps = 70;
const int trackLength = 1750;
const char* stageMessages [] = {"Keine_Stufe_eingestellt",
    "Stufe_1",
    "Stufe_2",
    "Stufe_3",
    "Stufe_4",
    "Stufe_5",
    "Stufe_6",
    "Stufe_7",
    "Stufe_8",
    "Stufe_9",
    "Stufe_10"};

const char* messages [] = {"Start ...",
    "Startbereit",
    "Programm_wird_ausgefuehrt ...",
    "Vorgang_abgebrochen ,_das_Geraet
.....muss_neu_gestartet_werden",
    ""};
```

[globale Variablen]globale Variablen

In der setup-Funktion werden die Startwerte für die maximale Geschwindigkeit und Beschleunigung des Schrittmotors gesetzt, die Pins für LEDs, Taster und Endschalter initialisiert, die serielle Kommunikation gestartet und das Display initialisiert. Ein Interrupt für den Encoder-Taster wird festgelegt. Das Display zeigt eine Startanimation, und die LEDs werden auf ihre Startzustände gesetzt. 3.4.2

```
void setup() {
    stepper.setMaxSpeed(1000);
    stepper.setAcceleration(1000);
    pinMode(PIN_LED_RED, OUTPUT);
    pinMode(PIN_LED_GREEN, OUTPUT);
    pinMode(PIN_LED_BLUE, OUTPUT);
    pinMode(PIN_BUTTON, INPUT_PULLUP);
    pinMode(PIN_ENDSTOP, INPUT_PULLUP);

    Serial.begin(9600);
    if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_WIDTH))
        Serial.println(F("SSD1306_allocation_failed"));
    for (;;)
    }

    attachInterrupt(digitalPinToInterrupt(PIN_SW), I
```

3 Schrittmotor

```
display.display();
delay(2000);
display.setTextSize(2);
display.setTextColor(SSD1306_WHITE);
updateDisplay(messages[0], messages[4]);
delay(1000);
updateLEDs(false, true, false);
}
```

[void setup]void setup

Die loop-Funktion wird kontinuierlich ausgeführt. Sie liest die Position des Encoders aus und zeigt bei einer Änderung die entsprechende Stufe auf dem Display an. Der Zustand des Tasters wird abgefragt und bei Betätigung die Referenzfahrt des Motors gestartet. Nach der Referenzfahrt führt der Motor Bewegungen gemäß den eingestellten Stufen und Geschwindigkeiten aus. 3.4.2

```
void loop() {
    long newPosition = encoder.read();
    if (newPosition != oldPosition) {
        stage = constrain(newPosition / 4, 1, 10);
        updateDisplay(messages[1], stageMessages[stage]);
        oldPosition = newPosition;
    }

    int buttonReading = digitalRead(PIN_BUTTON);
    if (buttonReading != lastButtonStatus) {
        lastDebounceTime = millis();
    }

    if ((millis() - lastDebounceTime) > debounceDelay) {
        if (buttonReading != buttonStatus) {
            buttonStatus = buttonReading;
            if (buttonStatus == LOW) {
                stepper.setAcceleration(acceleration);
                while (!findReference()) {
                    stepper.setSpeed(-speed);
                    stepper.runSpeed();
                }

                int currentStage = stage;
                updateDisplay(messages[2], stageMessages[currentStage]);
                updateLEDs(false, false, true);
                for (int i = 0; i < 2; i++) {
                    if (i == 0) {
                        stepper.setSpeed(speed);
                    }
                }
            }
        }
    }
}
```

3.4 Beschreibung des Programms auf dem Arduino

```

stepper.move(-tr
stepper.move(tr
while (s
}
}
else {
stepper.move(-tr
stepper.move(tr
stepper.move(tr
while (s
}
}
stepper.move(-tr
while (stepper.currentPosition() < tr
stepper.move(tr
}
}
updateDisplay(messages[1])
}
}
}
lastButtonStatus = buttonReading;
}

```

[void loop]void loop

Die updateLEDs-Funktion aktualisiert den Zustand der LEDs basierend auf den übergebenen Parametern. 3.4.2

```

void updateLEDs(bool red, bool green, bool blue) {
    digitalWrite(PIN_LED_RED, red ? HIGH : LOW);
    digitalWrite(PIN_LED_GREEN, green ? HIGH : LOW);
    digitalWrite(PIN_LED_BLUE, blue ? HIGH : LOW);
}

```

[void updateLEDs]void updateLEDs

Die updateDisplay-Funktion aktualisiert das Display mit den übergebenen Nachrichten. 3.4.2

```

void updateDisplay(const char* line1, const char* line2) {
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println(F(line1));
    if (line2 != "") {
        display.println(F(line2));
    }
}

```

3 Schrittmotor

```
        display.display();  
    }
```

[void updateDisplay]void updateDisplay

Die findReference-Funktion führt die Referenzfahrt aus und überprüft den Endschalter. Wenn der Endschalter gedrückt ist, stoppt der Motor und die Funktion gibt true zurück. 3.4.2

```
bool findReference() {  
    int endstopReading = digitalRead(PIN_ENDSTOP);  
    if (endstopReading != lastEndstopStatus) {  
        lastEndstopDebounceTime = millis();  
    }  
  
    if ((millis() - lastEndstopDebounceTime) > debounceTime) {  
        if (endstopReading != endstopStatus) {  
            endstopStatus = endstopReading;  
            if (endstopStatus == LOW) {  
                stepper.stop();  
                return true;  
            }  
        }  
    }  
    lastEndstopStatus = endstopReading;  
    return false;  
}
```

[bool findReference]bool findReference

Die handleInterrupt-Funktion wird aufgerufen, wenn der Encoder-Taster gedrückt wird. Sie stoppt den Motor und aktualisiert das Display mit einer Abbruchnachricht. 3.4.2

```
void handleInterrupt() {  
    stepper.stop();  
    updateDisplay(messages[3], messages[4]);  
}
```

[void handleInterrupt]void handleInterrupt

3.5 Bibliotheken

Für die Erstellung des Programms werden Bibliotheken verwendet, um die Programmierung zu vereinfachen.

3.5.1 Accelstepper

Die Arduino AccelStepper Bibliothek bietet eine objektorientierte Schnittstelle für die Steuerung von Schrittmotoren und Motortreibern über 2, 3 oder 4 Pins. Diese Bibliothek stellt eine wesentliche Verbesserung gegenüber der standardmäßigen Arduino Stepper Bibliothek dar und bietet zahlreiche erweiterte Funktionen, die für anspruchsvollere Anwendungen erforderlich sind. Sie wird in diesem Projekt in der Version 1.64 verwendet. Zu den herausragenden Merkmalen der AccelStepper Bibliothek gehört die Unterstützung von Beschleunigungs- und Verzögerungsprozessen. Dies ermöglicht eine feinere Steuerung der Motorbewegungen, was insbesondere bei präzisen Anwendungen von Vorteil ist. Darüber hinaus unterstützt die Bibliothek die gleichzeitige Steuerung mehrerer Schrittmotoren, wobei jeder Motor unabhängig voneinander gesteuert werden kann. Dies ist besonders nützlich in Anwendungen, die eine synchronisierte Bewegung mehrerer Motoren erfordern. Ein weiterer Vorteil der AccelStepper Bibliothek ist, dass die meisten API-Funktionen nicht blockieren oder verzögern, es sei denn, dies ist ausdrücklich angegeben. Dies trägt dazu bei, dass der Hauptprogrammfluss nicht unterbrochen wird, was die Effizienz und Reaktionsfähigkeit des Systems verbessert. Die Bibliothek unterstützt sowohl 2-, 3- als auch 4-Draht-Schrittmotoren sowie 3- und 4-Draht-Halbschrittmotoren. Darüber hinaus können alternative Schrittverfahren verwendet werden, um beispielsweise den AFMotor zu unterstützen. Die Theorie hinter der AccelStepper Bibliothek basiert auf den Geschwindigkeitsberechnungen, die in dem Artikel „Generate stepper-motor speed profiles in real time“ von David Austin beschrieben sind. Die Bibliothek verwendet Schritte pro Sekunde, anstelle von Radianen pro Sekunde, da der Schrittwinkel des Motors nicht bekannt ist. Ein anfängliches Schrittintervall wird basierend auf der gewünschten Beschleunigung berechnet, und bei nachfolgenden Schritten werden kürzere Schrittintervalle basierend auf dem vorherigen Schritt berechnet, bis die maximale Geschwindigkeit erreicht ist. Die AccelStepper Bibliothek wird unter einer freien GPL-Lizenz angeboten, was bedeutet, dass sie kostenlos genutzt und verändert werden kann, solange der Quellcode offengelegt wird. Für kommerzielle Anwendungen, bei denen der Quellcode nicht offengelegt werden soll, steht eine kommerzielle Lizenz zur Verfügung. Zusammenfassend bietet die Arduino AccelStepper Bibliothek eine leistungsstarke und flexible Lösung für die Steuerung von Schrittmotoren. Mit ihrer Unterstützung für Beschleunigung und Verzögerung, der gleichzeitigen Steuerung mehrerer Motoren und ihrer nicht blockierenden API ist sie eine ausgezeichnete Wahl für anspruchsvolle Anwendungen.[McCauley:2024; McCauley:2024b]

3.5.2 Wire

Die Wire Bibliothek ermöglicht die Kommunikation mit I2C-Geräten auf allen Arduino-Plattformen und ist ein weit verbreitetes Protokoll zum Lesen und Senden von Daten zu und von externen I2C-Komponenten. Die I2C-Pins variieren je nach Hardware-Design und Architektur der verschiedenen Arduino-Boards, wobei die Standard-I2C-Pins für Nano Boards die Pins A4 (SDA) und A5 (SDL) sind.

3.5.3 Adafruit GFX

Die Adafruit GFX Bibliothek ist eine Kern-Grafikbibliothek, die grundlegende Grafikprimitiven wie Punkte, Linien und Kreise für Adafruit-Displays bereitstellt. Sie ist unerlässlich für die Darstellung von Grafiken auf verschiedenen Hardware-Displays und erfordert für jedes Display eine spezifische Hardware-Bibliothek zur Verwaltung der niedrigeren Funktionen. Zusätzlich unterstützt sie verschiedene Bitmap-Schriftarten und bietet Werkzeuge wie Image2Code zur Konvertierung von BMP-Dateien in Code-Arrays für die Anzeige. Die Bibliothek legt besonderen Wert auf die Rückwärtskompatibilität mit bestehenden Arduino-Sketches und bietet umfassende Möglichkeiten zur Optimierung und Anpassung von Grafik- und Textdarstellungen auf den Displays. Sie wird in diesem Projekt in der Version 1.11.9 verwendet.[AdafruitIndustries:2024]

3.5.4 Adafruit SSD1306

Die Adafruit SSD1306 Bibliothek ist speziell für monochrome OLED-Displays entwickelt, die den SSD1306 Treiber verwenden. Diese Displays kommunizieren über I2C oder SPI und erfordern 2 bis 5 Pins für die Verbindung. Die Bibliothek bietet grundlegende Funktionen zur Steuerung der Displays wie das Initialisieren der Kommunikation, das Zeichnen von Grafikelementen wie Linien und Kreisen sowie das Anzeigen von Text. Sie ermöglicht die einfache Anpassung der Displaygröße über den Konstruktor und unterstützt sowohl SPI-Transaktionen als auch die Spezifikation der SPI-Bitrate ab Arduino 1.6 oder höher. Die Installation erfolgt idealerweise über den Arduino IDE Bibliotheksmanager oder durch manuelles Herunterladen und Entpacken des Quellcodes von GitHub. Sie wird in diesem Projekt in der Version 2.5.10 verwendet.[AdafruitIndustries:2024b]

3.5.5 Encoder

Die Encoder Library ermöglicht das Zählen von Impulsen aus bestimmten Signalen, die häufig von Drehknöpfen, Motor- oder Wellensensoren sowie anderen Positionsgebern stammen. Diese Bibliothek ist für Arduino und Teensy Boards optimiert und bietet verschiedene Zählmodi, darunter 4X counting für präzise Positionserfassung. Die Verwendung erfolgt durch die Initialisierung eines Encoder-Objekts mit zwei Pins, wobei der erste Pin idealerweise über Interruptfähigkeit verfügt für optimale Leistung. Die Bibliothek unterstützt sowohl I2C- als auch SPI-Schnittstellen und bietet verschiedene Hardware- und Softwareoptionen zur Anpassung an die Anforderungen des Benutzers. Sie wird in diesem Projekt in der Version 1.4.4 verwendet.[Stoffregen:2024]

3.6 Schrittmotorsteuerung

Für eine einfachere Bedienung des Schrittmotors wurde ein Schrittmotortreiber mit integriertem Übersetzer verbaut. Bei einer Leistung von bis zu 35 V und ± 2 A ist der DEBO DRV A4988 für den Betrieb von bipolaren Schrittmotoren ausgelegt. Auf

dem Treiber ist ein Stromregler mit fester Ausschaltzeit integriert, bei dem zwischen einem langsamen und einem gemischten Abschaltmodus gewählt werden kann. Durch eine Impulseingabe wird der Motor um einen Mikroschritt angetrieben. Der Vorteil der Schrittmotorsteuerung liegt darin, dass keine Phasensequenztabellen, Hochfrequenz-Steuerleitungen oder komplexe Schnittstellen programmiert werden müssen. Im Schrittbetrieb wählt der A4988 automatisch den Abklingmodus, langsam oder gemischt. Im gemischten Modus wird im Abklingvorgang zwischen dem schnellen und langsamen Modus gewechselt. Dabei führt der gemischte Modus zu einer Verringerung der hörbaren Motorgeräusche, einer höheren Schrittnauigkeit und einer geringeren Verlustleistung. [Allegro:2022; Allegro:2021]

Abbildung 3.6: Treiberkarte Debo DRV A4988

Weitere Details:

- **Abmessungen** ($b \times l \times h$): $5\text{ mm} \times 5\text{ mm} \times 0,9\text{ mm}$
- **Steuerspannung**: 3,3 - 5 V
- **Maximale Ausgangskapazität**: bis 35 V und $\mp 2\text{ A}$
- **Schutzschaltungen**: Thermische Abschaltfunktion, Schutz vor Masseschluss, Schutz vor kurzgeschlossener Last, Überkreuzungsstromschutz [Allegro:2022]
- Das Board auf Basis des A4988 Treibers für Schrittmotoren bietet Auflösungen bis zu 1/16 Schritten.
- Die Auflösung kann manuell über einen DIP-Schalter gewählt werden.
- Richtungswahl
- einstellbare Stromstärke
- intelligente Chopping-Control
- Unterspannungs-, Übertemperaturschutz

3.7 Schrittlverluste verhindern

Eine große Herausforderung, bei der Verwendung von Schrittmotoren, stellen Schrittlverluste da. Um mögliche Schrittlverlusten einzugrenzen und oder sie zu verhindern, wird in diesem Kapitel beschrieben, wie die Ursachen für Schrittlverluste oder Stillstand methodisch zu ermitteln sind. [Faulhaber:2020]

3.8 Auswahl des Schrittmotors

Zunächst muss ein passender Motor für die Anwendung gewählt werden. Dabei sollten folgende grundlegende Regeln erfüllt sein:

- Motorauswahl durch Höchstwerte für Drehmoment und Drehzahl (Worst-Case-Szenario)
- Verwendung eines Sicherheitsaufschlag von 30 % auf die Drehmoment-Drehzahl-Kennlinie(Kippmoment)
- Sicherstellen, dass externe Ereignisse die Anwendung nicht blockieren können

Sind die geforderten Drehmomente bei den jeweiligen Drehzahlen, den Motorspezifikation entsprechend, dann sind keine Probleme zu erwarten. Ist der Motor zu schwach gewählt und die Anwendung fordert mehr Leistung als der Motor abgeben kann, so bleibt der Motor stehen. Der nächste Schritt ist die Durchführung von Testdurchläufen. Es soll im Betrieb überprüft werden, ob Schrittwverluste auftreten. Schrittmotoren verlieren konstruktionsbedingt nicht nur einen einzigen Schritt. Bei geringen Drehzahlen verliert der Motor ein Vielfaches von vier Schritten.[Faulhaber:2020]

3.9 Betriebsart

In diesem Abschnitt werden je nach Betriebsart mögliche Ursachen erläutert, falls der Schrittmotor bei den Tests versagt.

3.9.1 Start-Stopp-Betrieb

Der Motor ist mit der Last fest verbunden und wird mit konstanter Drehzahl betrieben. Innerhalb des ersten Schrittes muss der Motor auf die vorgegebene Frequenz beschleunigen.[Faulhaber:2020]

Fehlerbild: Motor läuft nicht an

Ursachen	Lösungen
Last zu Hoch	Falscher Motor, größeren Motor wählen
Frequenz zu hoch	Frequenz reduzieren
Pendelt der Motor von Links nach Rechts	Es könnte eine Phase unterbrochen oder nicht angeschlossen sein, dies muss repariert werden
Phasenstrom passt nicht	Phasenstrom erhöhen

Tabelle 3.4: Ursachen und Lösungen: Motor läuft nicht an [Faulhaber:2020]

3.9.2 Beschleunigung und Rampenprofil (Trapezförmig)

Der Motor kann mit einer im Controller vorgegebenen Beschleunigungsrate bis auf die Maximalfrequenz beschleunigen.[Faulhaber:2020]

Fehlerbild: Motor läuft nicht an (siehe Ursachen und Lösungen aus Tabelle 3.1 Ursachen und Lösungen: Motor läuft nicht an [Faulhaber:2020])

Fehlerbild: Motor beendet die Beschleunigungsrampe nicht

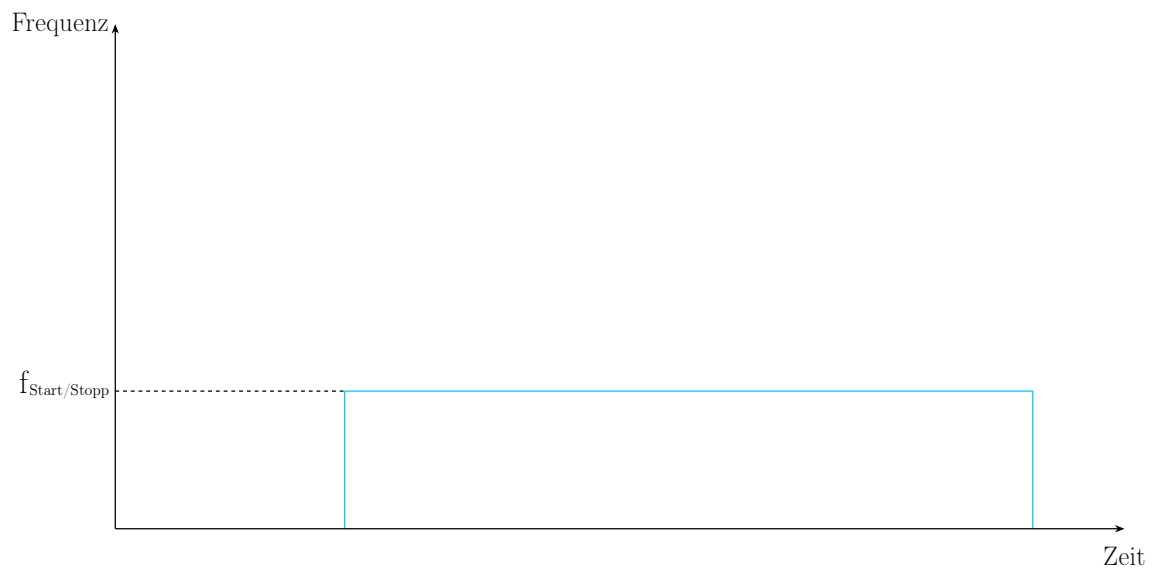


Abbildung 3.7: Start-Stopp Frequenz [Faulhaber:2020]

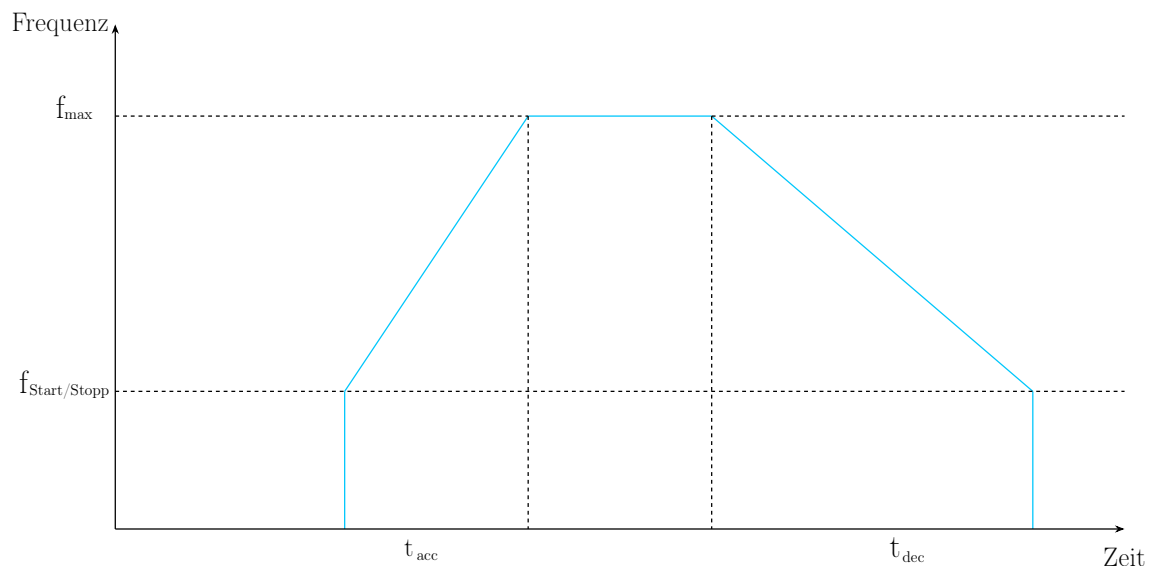


Abbildung 3.8: Trapezförmiges Geschwindigkeitsprofil [Faulhaber:2020]

Ursachen	Lösungen
Motor bleibt bei der Resonanzfrequenz hängen	• Beschleunigung erhöhen, um die Resonanzfrequenz schneller zu durchlaufen • Start-Stopp Frequenz über dem Resonanzpunkt wählen • Halbschritt- oder Mikroschrittbetrieb verwenden • Mechanische Dämpfung vorsehen
Falsche Einstellung von Versorgungsspannung oder Strom zu gering	• Spannung oder Strom erhöhen • Motor mit geringerer Impedanz testen • Stromregelung verwenden
Maximaldrehzahl zu hoch	• Maximaldrehzahl reduzieren • Beschleunigungsrampe abflachen
Schlechte Vorgabe der Beschleunigungsrampe durch	

3 Schrittmotor

Tabelle 3.5: Ursachen und Lösungen: Motor beendet die Beschleunigungsrampe nicht [Faulhaber:2020]

Fehlerbild: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist.

Ursachen	Lösungen
Motor wird an Leistungsgrenze betrieben und bleibt stehen aufgrund zu hoher Beschleunigung	• Ruckeln verringern durch geringere Beschleunigungsrate oder durch unterschiedliche Beschleunigungsrampen, erst steil dann flacher • Drehmoment erhöhen • Motor im Mikroschrittbetrieb betreiben • Mechanische Dämpfung vorsehen

Tabelle 3.6: Ursachen und Lösungen: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist. [Faulhaber:2020]

3.10 Externe Ereignisse

3.10.1 Lastrückkopplung

Manchmal wird der vom Motor angetriebene Mechanismus/Last während der Bewegung „aufgezogen“ und gibt diese Energie wieder an den Motor zurück, wenn die Ströme ausgeschaltet werden. Der Mechanismus könnte z.B. ein Untersetzungsgetriebe sein. [Faulhaber:2020] Wird diese Energie an den Motor zurück geleitet, kann es passieren, dass der Motor sich um einen Winkel verdreht, der mehr als einem Schritt entspricht. Dabei kann es passieren, dass der Motor nicht ausreichend Drehmoment entwickelt und nicht oder erst nach 4 Vollschritten anläuft. [Faulhaber:2020]

Lösungen:

- Die Kommutierung so programmieren, dass Wert und Polarität vor dem Abschalten gespeichert und beim Wiedereinschalten verwendet werden kann.
- Nicht vollständig den Strom abschalten, sondern bei Motorstillstand einen reduzierten *Stand-By* Strom aufrechterhalten

3.10.2 Erhöhung der Nutzlast mit der Zeit

Manchmal läuft der Motor für eine lange Zeit störungsfrei und viel später treten die ersten Schrittverluste auf. In diesem Fall ist es sehr wahrscheinlich, dass die Last, die der Motor „sieht“, sich geändert hat. Das kann auf Verschleiß der Motorlager oder ein externes Ereignis zurückzuführen sein. [Faulhaber:2020]

Lösungen:

3.10 Externe Ereignisse

- Prüfen ob ein externes Ereignis durch Veränderung des Mechanismus vorliegt.
- Prüfen ob Lagerverschleiß vorhanden ist. Verwendung von Kugellager erhöhen die Lebensdauer des Motors.
- Verwendung von Schmiermittel um Reibung zu verhindern.

4 Skizze

5 Liste der Teile

6 Aufbau

7 Funktionen

hier kommen Funktionen von unserem

8 Wartung

9 Sicherheitshinweise

10 Troubleshooting

11 Sonstiges

